動画の Mamba は、過去の情報を
何に紐付け、どこまで信じるか

MambaOut [CVPR'25] / TV3S [CVPR'25] / TS-Mamba [ICLR'26] / Memory as Dynamics [ICML'26]

油谷陽樹（名工大 玉木・丁研）

2026/11/--（見本）

本紹介の概要：過去の情報を何に紐付け、どこまで信じるか

■ 背景

- 動画を1フレームずつ処理するモデルは、過去のフレームの情報をどこかに持ち、使う必要がある
- Mamba（SSM）は系列を**固定サイズの state** に圧縮しながら読む。動画では時間方向の系列にも使える

■ 今日の問い

- ① 過去の情報を**何に紐付けるか**（位置・軌跡・物体）
- ② それを**どこまで信じるか**（無条件・類似度で選ぶ・信頼度で 0/1 に切り替える）

MambaOut
前提：Mamba は要るか

→ **TV3S**
位置に紐付けて
持ち越す

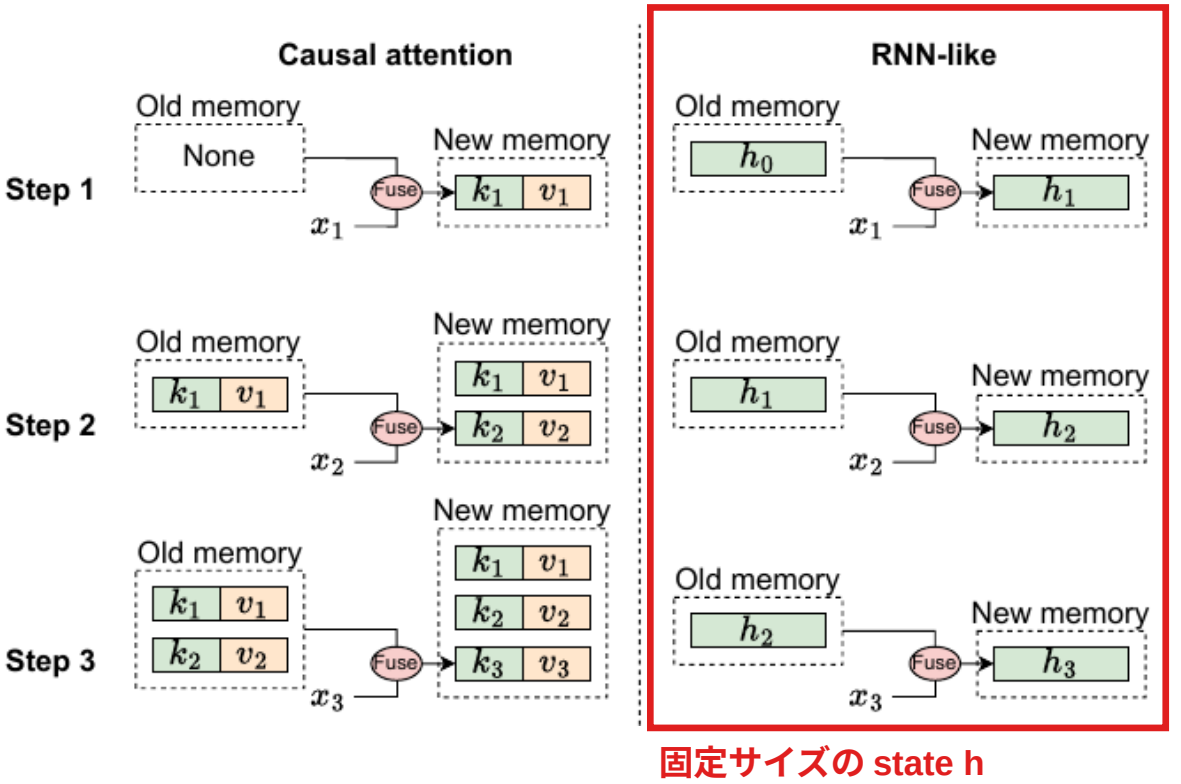
→ **TS-Mamba**
軌跡に沿って
集める

→ **Memory as Dynamics**
物体の予測を
信頼度で選ぶ

MambaOut で「Mamba が向く系列」を押さえ、残りの3本で紐付け先と信じ方の違いを見る
※ 3本はタスクが違うので、優劣ではなく設計の違いとして比べる

事前知識：SSM の state は「固定サイズで、情報を落とす memory」

- SSM（状態空間モデル）
  - $h_t = \bar{A}\, h_{t-1} + \bar{B}\, x_t \qquad y_t = C\, h_t$
  - state h に過去をすべて押し込み、毎ステップ更新
- Mamba（Selective SSM）
  - Δ・B・C を入力から決める → 何を残すかを入力に応じて選べる
- attention との違い（右図）
  - attention：過去の key/value を全部持つ（lossless、系列長とともにコスト増）
  - SSM：サイズ一定（**lossy**、コストは一定）



固定サイズの state h

式と図：MambaOut [Yu+, CVPR2025] Eq.(1)–(3), Fig. 2

MambaOut: Do We Really Need Mamba for Vision?

前提：画像認識に Mamba は要るか？ SSM を抜いて確かめる

Weihao Yu, Xinchao Wang

CVPR 2025

MambaOut
前提：Mamba は要るか

TV3S
位置に紐付けて
持ち越す

TS-Mamba
軌跡に沿って
集める

Memory as Dynamics
物体の予測を
信頼度で選ぶ

問い：Mamba が向くのは「長い系列」かつ「causal」なタスク

■ 著者の整理（Sec. 3.1）

・特性1：長い系列を大域的に扱う（state は lossy だが、長い系列でもコスト一定）

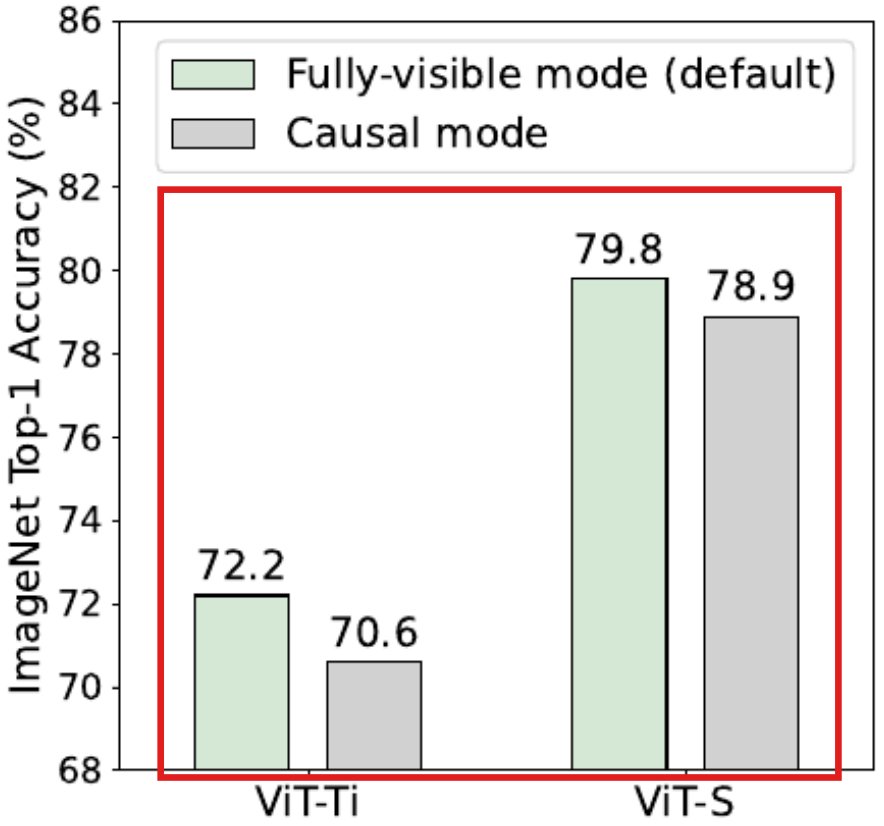
・特性2：causal な token mixing が必要（SSM は本質的に causal）

■ 「長い」の目安：L > 6D（L：token 数、D：channel 数）

・ViT-S なら 6 × 384 = 2304

タスク	token 数	長い？	causal？
ImageNet 分類（224²）	196	×	×
COCO 検出・ADE20K	約 4K	○	×

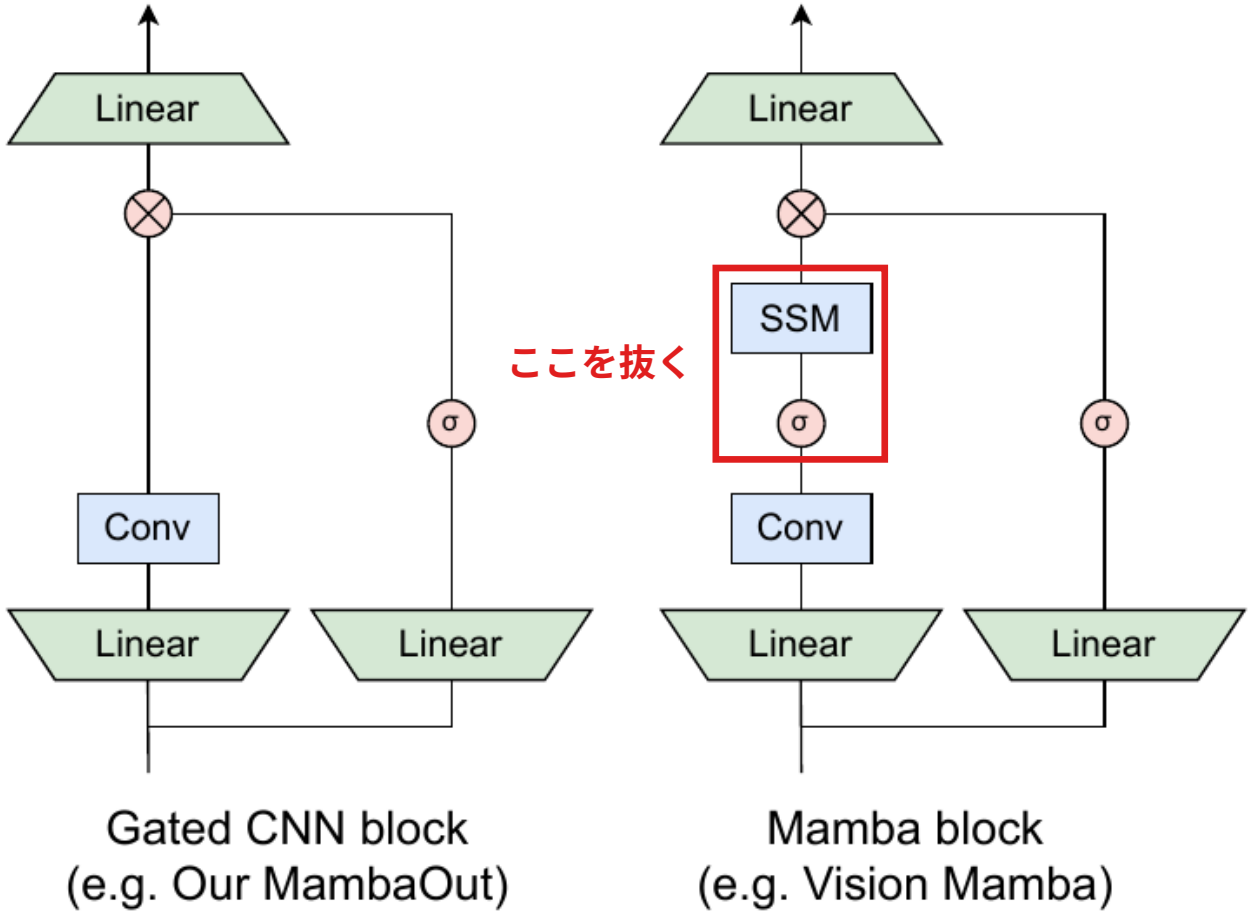
→ 仮説1：分類に SSM は不要　／　仮説2：検出・セグメンテーションでは検討の余地あり



ViT を causal にすると精度が下がる
72.2 → 70.6（Ti）、79.8 → 78.9（S）

MambaOut Sec. 3.1–3.4, Fig. 3(b)

手法：MambaOut ＝ Mamba block から SSM を抜いた Gated CNN



■ token mixer の比較（Eq. 10, 11）

- Gated CNN：Conv(Z)
- Mamba：SSM(σ(Conv(Z)))
- 違いは **SSM（と直前の σ）** だけ

■ つまりやっていること

- 「Mamba から SSM を抜いても同じくらい性能が出るなら、そのタスクに SSM は要らない」

■ 実装

- token mixer は 7×7 depthwise conv、4 stage 構成

MambaOut Fig. 1(a), Eq. (10)(11), Sec. 4.1

結果：分類では SSM なしでも並ぶか上回り、検出では届かない

Model	Token Mixing Type	Param (M)	Test@224^2 MAC (G)	Test@224^2 Acc (%)
VMamba-S [56]	Conv + SSM	44	11.2	83.5
LocalVMamba-S [41]	Conv + SSM	50	11.4	83.7
PlainMamba-L2 [96]	Conv + SSM	25	8.1	81.6
VMambaV3-S* [55]	Conv + SSM	50	8.7	83.6
MambaOut-Small	Conv	48	9.0	**84.1**

検出・セグメンテーション（Tiny、Table 2・3 から抜粋）

Backbone	COCO AP^b	COCO AP^m	ADE20K mIoU（SS）
VMamba-T	46.5	42.1	47.3
LocalVMamba-T	46.7	42.2	47.9
MambaOut-Tiny（SSMなし）	45.1	41.0	47.4

MambaOut Table 1（ImageNet）, Table 2（COCO）, Table 3（ADE20K）

- ImageNet（仮説1）
  - MambaOut-Small **84.1**% vs LocalVMamba-S 83.7%（MACs は 79%）
  - Tiny は 82.7% で同値（LocalVMamba-T）
- COCO / ADE20K（仮説2）
  - MambaOut-Tiny は VMamba-T に AP^b −1.4、AP^m −1.1
  - ADE20K は LocalVMamba-T に −0.5 mIoU
  - ただし差は小さく、相手によっては MambaOut が上

まとめ：SSM は「長く、causal な系列」で使う



- MambaOut
  - SSM の state は固定サイズで lossy。長い系列でもコストが一定なのが強み
  - 分類（短い・非 causal）では SSM は不要。検出・セグメンテーションは検討の余地あり
- 限界
  - Tiny では同値（82.7%）。検出での差も 1 ポイント前後で、SSM ありが勝つ理由は切り分けられていない
- 動画に当てはめると（発表者の解釈。論文は動画を扱っていない）
  - オンライン動画の時間方向は、**長く、causal**（未来フレームは見られない）→ 2つの特性を両方満たしうる

次論文：動画の時間方向で、過去の情報をどう持つか？
→ 位置に紐付けて持ち越す（TV3S）

Exploiting Temporal State Space Sharing for Video Semantic Segmentation（TV3S）

過去の情報を位置に紐付けて持ち越す動画セグメンテーション

Syed Ariff Syed Hesham, Yun Liu, Guolei Sun, Henghui Ding, Jing Yang, Ender Konukoglu, Xue Geng, Xudong Jiang

CVPR 2025

MambaOut
前提：Mamba は要るか

TV3S
位置に紐付けて
持ち越す

TS-Mamba
軌跡に沿って
集める

Memory as Dynamics
物体の予測を
信頼度で選ぶ

概要：過去フレームの feature pool の代わりに、state を持ち越す

■ タスク：Video Semantic Segmentation（VSPW など）

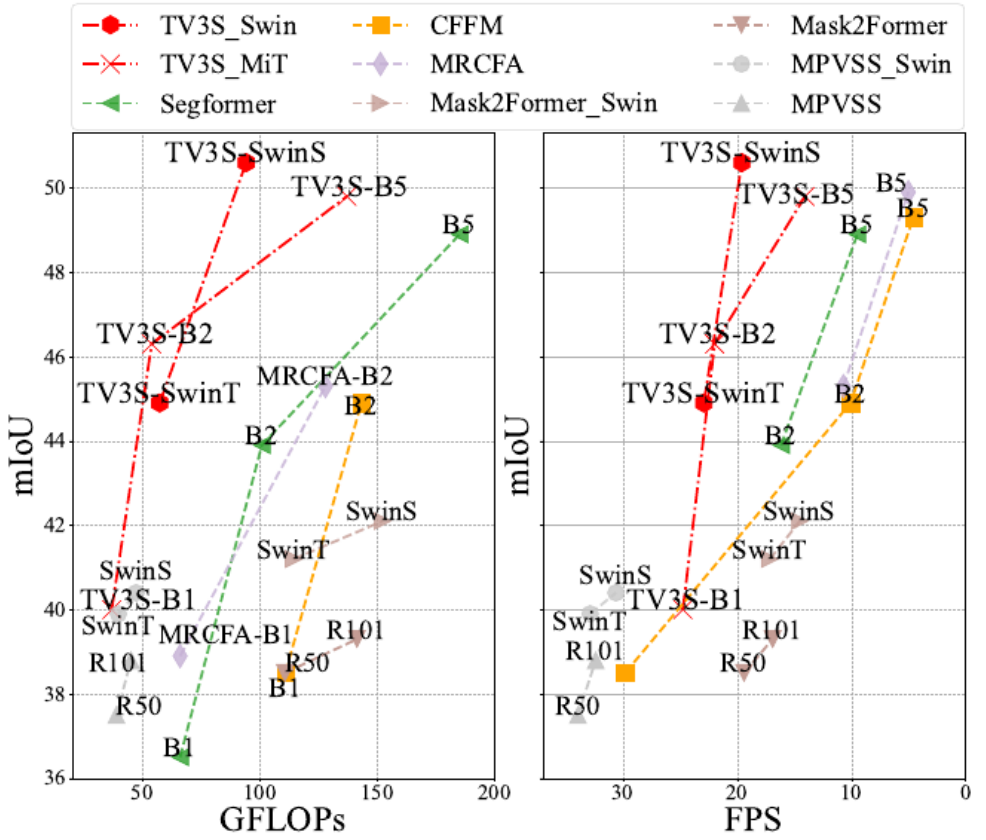
■ 前提（著者）：連続するフレームの間の変化は小さい → window の位置を固定したまま処理できる（Sec. 3.2）

■ 従来手法の課題

- 1フレームずつ or 短い窓で処理 → 時間方向の文脈が短い
- 過去フレームの特徴を保持（feature pool）→ メモリが重い

■ 提案：TV3S

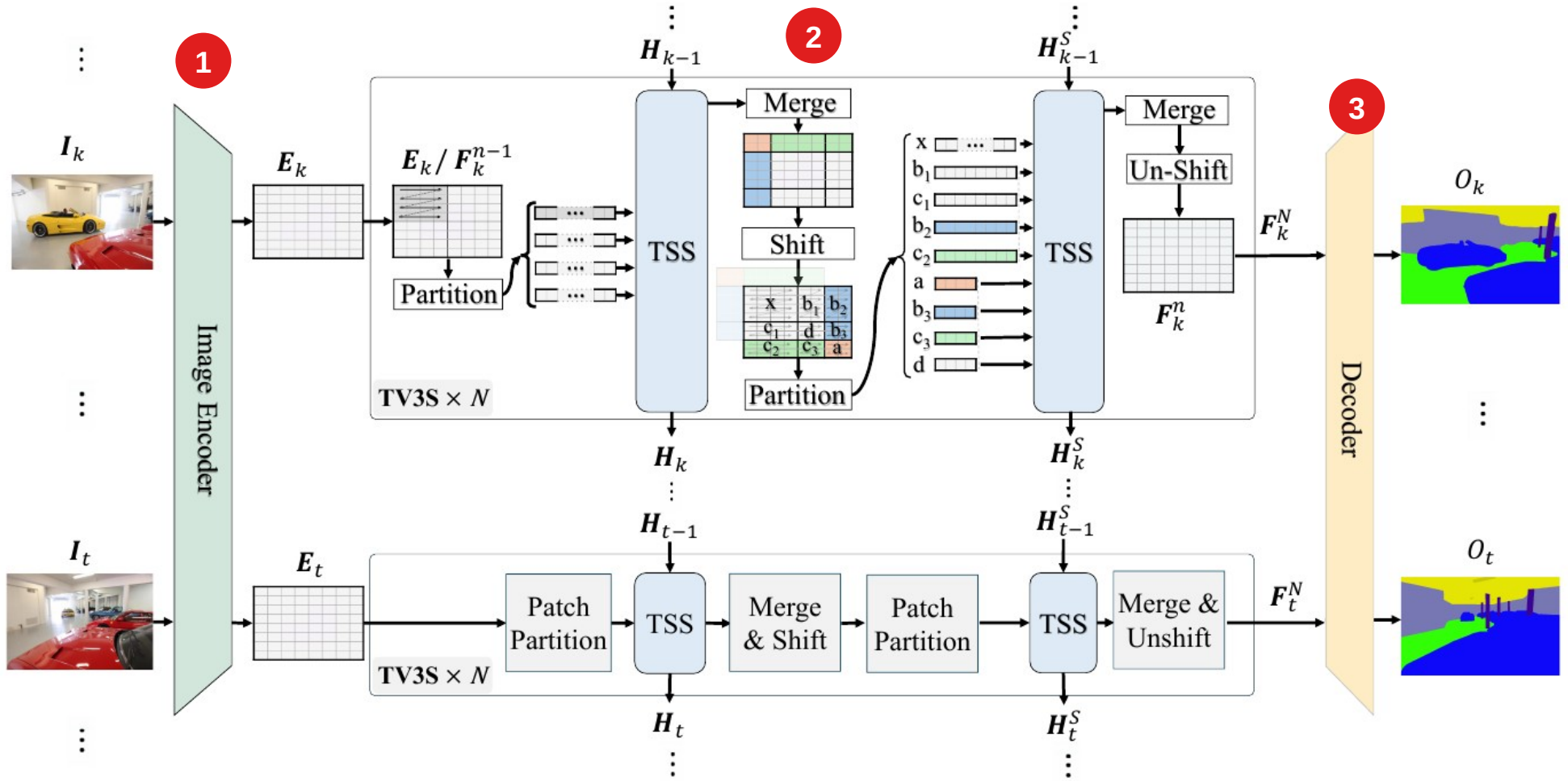
- 空間を window に分け、window ごとに Mamba の state を**フレーム間で持ち越す**
- shift で隣の window の情報も混ぜる。推論はオンライン



赤い線が TV3S：同程度の計算量・速度で mIoU が高い（VSPW）

TV3S [Hesham+, CVPR2025] Abstract, Sec. 1, Sec. 3.2, Fig. 1

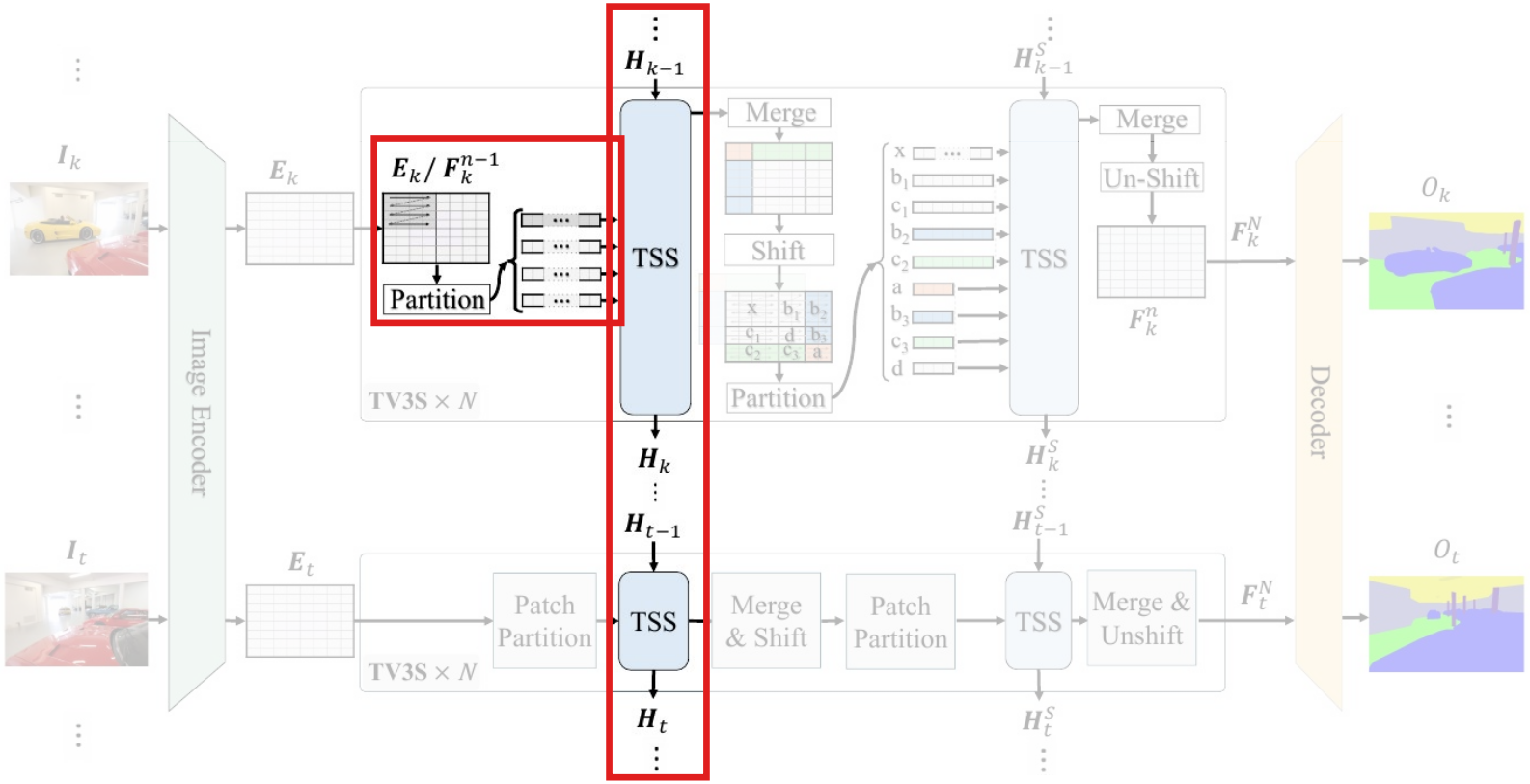
全体図：encoder → TV3S block × 4 → 分類



① 画像 encoder（MiT など）　② TV3S block：window ごとの Mamba（上段が詳細、下段が次フレーム）　③ 線形層で分類

TV3S Fig. 2

state は 20×20 の window ごとに1つ、位置に固定して持ち越す

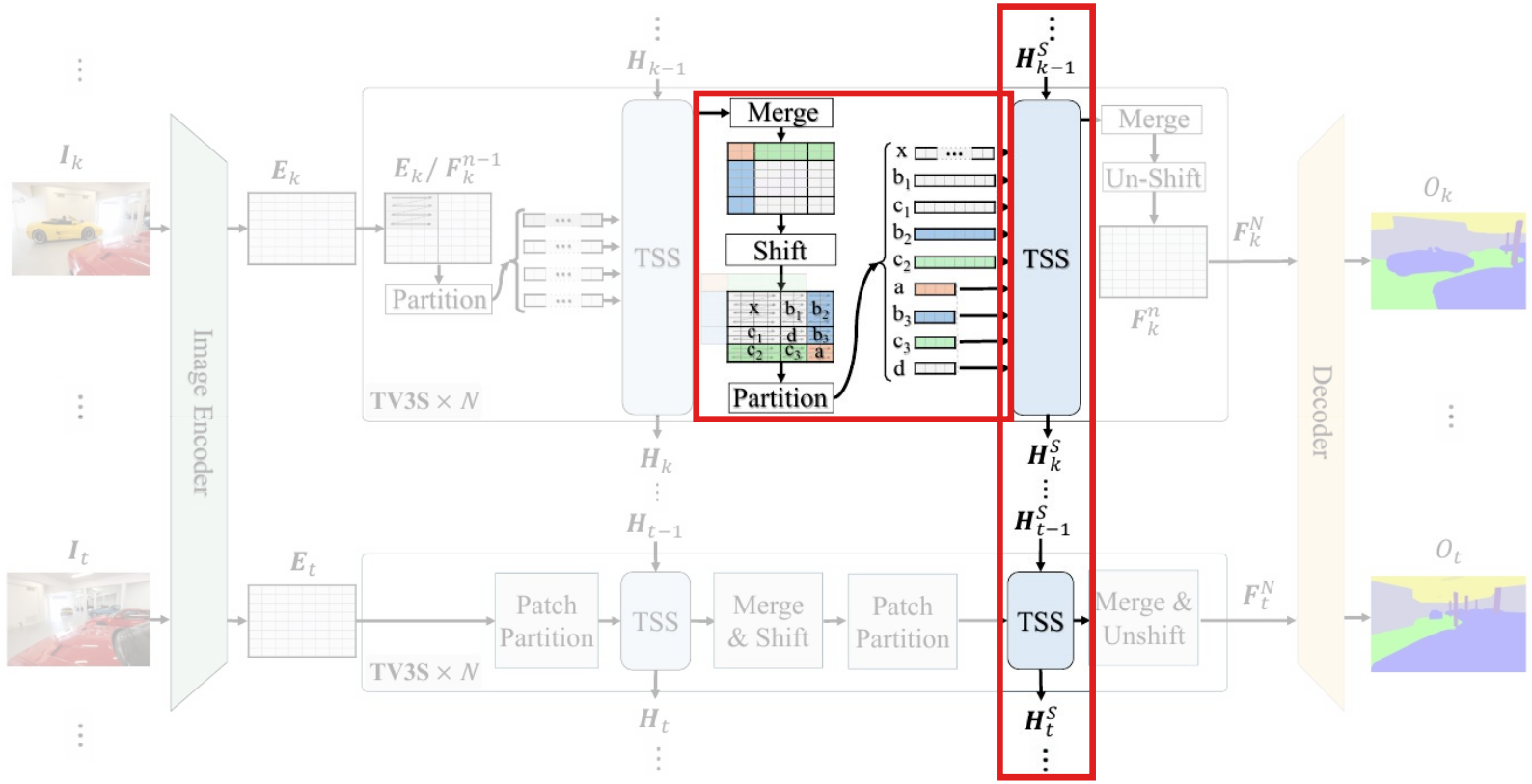


$$\boldsymbol{H}_t^{i,j} = f_A(\Delta, \boldsymbol{A}_s)\boldsymbol{H}_{t-1}^{i,j} + f_B(\Delta, \boldsymbol{A}_s, \boldsymbol{B}_s)\hat{\boldsymbol{P}}_t^{i,j}, \quad (2)$$
$$\boldsymbol{F}_t^{i,j} = \boldsymbol{C}_s\boldsymbol{H}_t^{i,j}.$$

- 20×20 の window に分割
  - 公開コードの設定では入力の 1/8 解像度 → 入力画像で約 160×160 画素
- window の中の token を1列に並べて scan し（Eq. 1）、window ごとの state $H^{i,j}$ を次フレームへ渡す
  - state の数は (W/w) × (H/w) 個
- 動きに合わせて window を動かす仕組みはない（著者の前提：フレーム間の変化は小さい）

TV3S Fig. 2, Eq. (1)(2), Sec. 3.2, 公開コード（window の画素数）

shift：window の境界をずらし、同じフレーム内で隣と混ぜる



■ Swin と同じく、window を 10 ずらして分け直す

■ ずらした側の window にも別の state H^S を持たせる

■ 混ぜるのは**同じフレーム内の空間方向**

- 時間方向の位置合わせではない

■ 効果：mIoU 38.9 → 39.5（+0.6、Table 3。補足）

つまり：位置に固定した state が、隣の window の情報も少しずつ受け取れるようにする

TV3S Fig. 2, Sec. 3.2 "Shifting and edge handling", Table 3

実験：持ち越す過去が長いほど、時間方向の一貫性（mVC）が伸びる

Number of Frames	mIoU	mVC_8	mVC_{16}
1	37.9	84.5	79.1
2	39.2	86.9	81.8
4	39.5	88.9	84.2
8	39.7	90.4	85.9
16	39.7	91.2	87.2
32	**39.7**	**91.5**	**87.8**

mVC_n：連続 n フレームで正解と一致し続ける領域の割合
（ちらつきが少ないほど高い）

精度（mIoU）はすぐ飽和するが、
ちらつきの少なさ（mVC）は長く持ち越すほど伸びる

■ 推論時に使う過去フレーム数を変える（VSPW、MiT-B1）

- mVC_{16}：79.1 → **87.8**（1 → 32フレーム）
- mIoU：37.9 → 39.7、8フレームで頭打ち

■ state を外して学習し直した比較はない。評価条件も Table 1 と違う

TV3S Table 6, Sec. 4.1, Eq. (4), Sec. 4.4

結果：B1 では mIoU・mVC とも最高、B5 では mIoU のみ2位

Methods	Backbones	mIoU↑	mVC_8↑	mVC_{16}↑	GFLOPs↓	Params(M)↓	FPS↑
Segformer† [69]	MiT-B1	36.5	84.7	79.9	26.6	13.8	58.7
CFFM [57]	MiT-B1	38.5	88.6	84.1	-	15.5	29.8
MRCFA [58]	MiT-B1	38.9	88.8	84.4	-	16.2	40.1
TV3S (Ours)	MiT-B1	**40.0**	**90.7**	**87.0**	36.9	17.3	24.7
Segformer [69]	MiT-B5	48.9	87.8	83.7	185.0	82.1	9.4
CFFM [57]	MiT-B5	49.3	90.8	87.1	413.5	85.5	4.5
MRCFA [58]	MiT-B5	**49.9**	90.9	87.4	373.0	84.5	5.0
TV3S (Ours)	MiT-B5	49.8	**91.7**	**88.7**	137.0	85.6	14.0

- MiT-B1：TV3S **40.0** / 90.7 / 87.0（mIoU / mVC8 / mVC16）、MRCFA 38.9 / 88.8 / 84.4
- MiT-B5：mIoU は MRCFA 49.9 に 0.1 届かず2位（49.8）。mVC と速度（14.0 FPS vs 5.0）は上
- feature pool を持つ手法より計算量が小さい（MiT-B5：137.0 vs 373.0 GFLOPs）

TV3S Table 1（VSPW）

まとめ：位置に固定した state を、隣と混ぜながら持ち越す



- TV3S
  - 20×20 の window ごとに Mamba の state を持ち、フレーム間で持ち越す
  - shift で同じフレーム内の隣の window と混ぜる
  - 長く持ち越すほど時間方向の一貫性（mVC）が上がる
- 限界
  - mIoU は8フレームで頭打ち。shift の改善も +0.6 と小さい
  - window は位置に固定（フレーム間の変化は小さいという前提）
  - 学習（4フレーム）と推論（動画全体）で系列長が違う
  - Mamba を他の時系列モデルに替えた比較はない

次論文：位置に固定せず、動きに沿って過去を集めたら？
→ TS-Mamba（state は持ち越さず、毎フレーム集め直す）

Trajectory-aware Shifted State Space Models for Online Video Super-Resolution（TS-Mamba）

動きの軌跡に沿って過去を集めるオンライン動画超解像

Qiang Zhu, Xiandong Meng, Yuxuan Jiang, Fan Zhang, David Bull, Shuyuan Zhu, Bing Zeng, Ronggang Wang

ICLR 2026

MambaOut
前提：Mamba は要るか

TV3S
位置に紐付けて
持ち越す

TS-Mamba
軌跡に沿って
集める

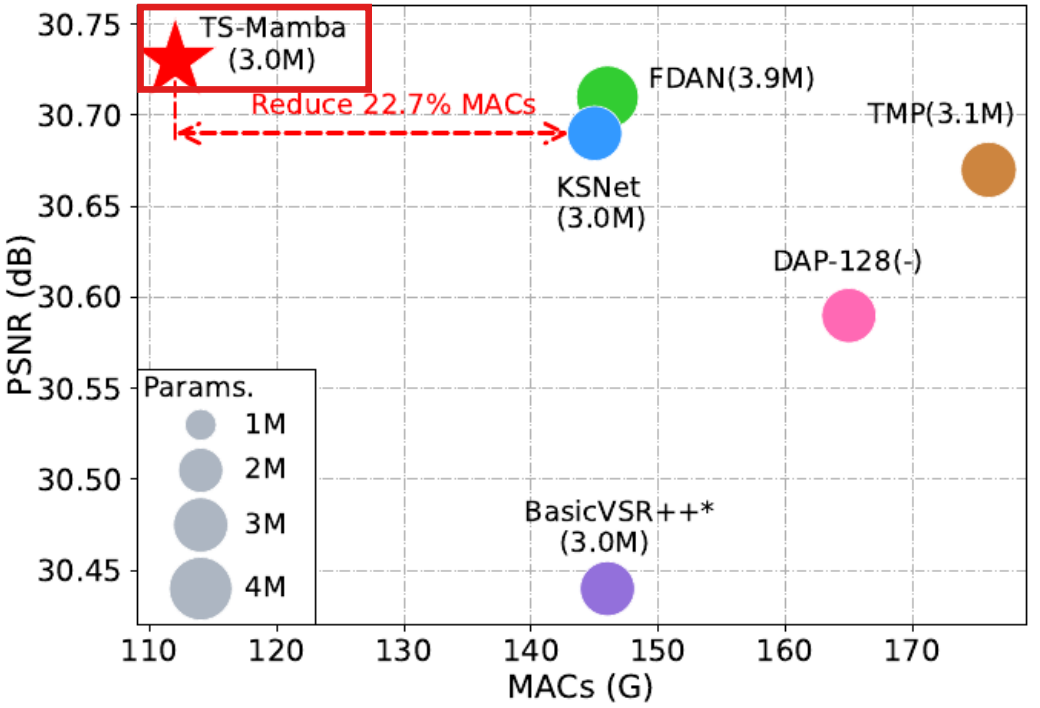
Memory as Dynamics
物体の予測を
信頼度で選ぶ

概要：軌跡に沿って過去15フレームから似た token を選び、Mamba で集約

■ タスク：オンライン動画超解像（VSR）
- 現在と**過去**のフレームだけで高解像度化（未来は使わない）

■ 従来のオンライン VSR の課題
- 計算量の制約で、位置合わせ（alignment）に使うのは直前1フレームだけ

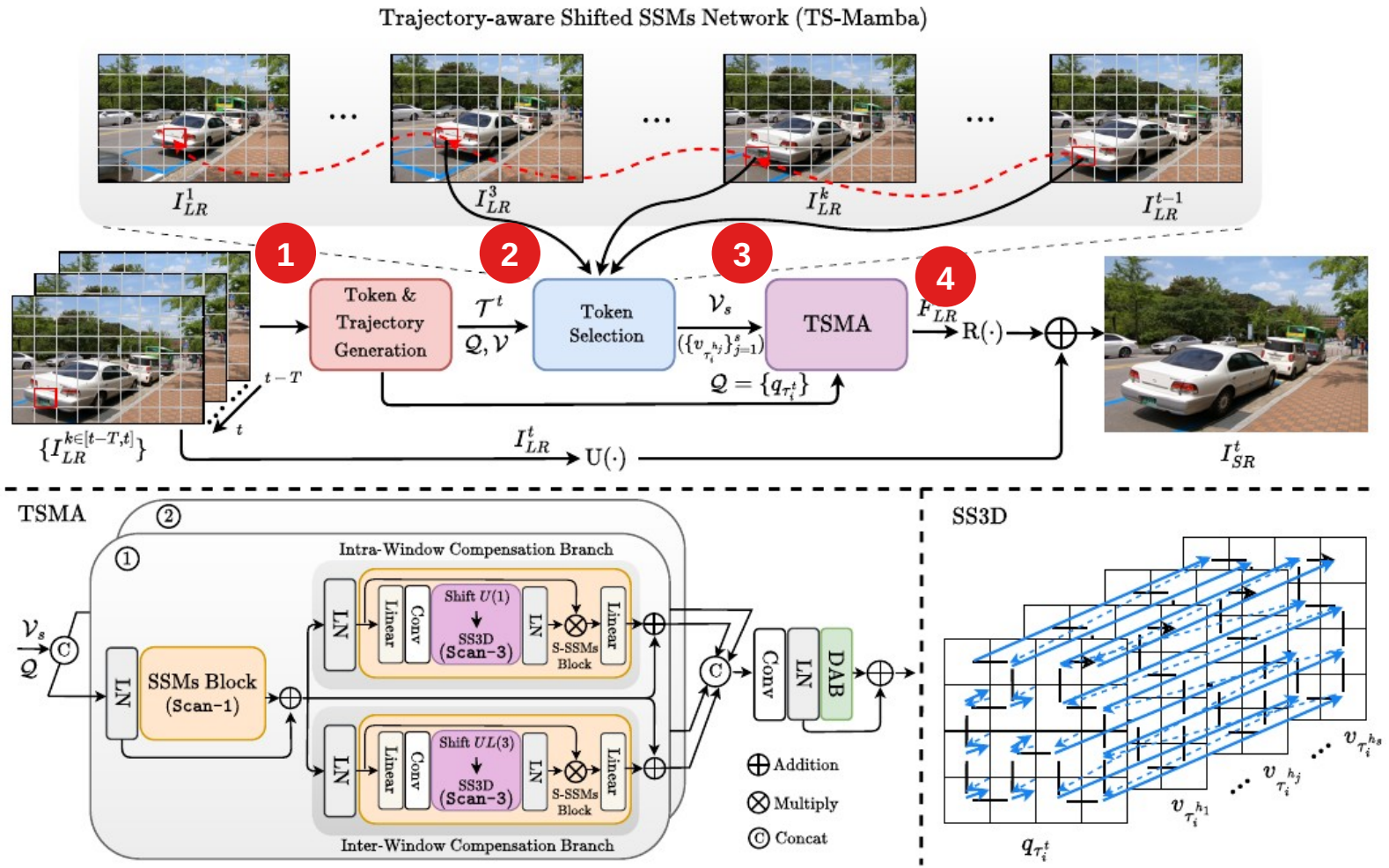
■ 提案：TS-Mamba
- 軌跡に沿って過去の token を選び、Mamba でまとめる
- 1列に並べたときの切れ目は、shift と Hilbert scan で補う



赤枠が TS-Mamba：PSNR が高く、MACs が小さい（REDS4）

TS-Mamba [Zhu+, ICLR2026] Sec. 1, Sec. 2.2, Fig. 1

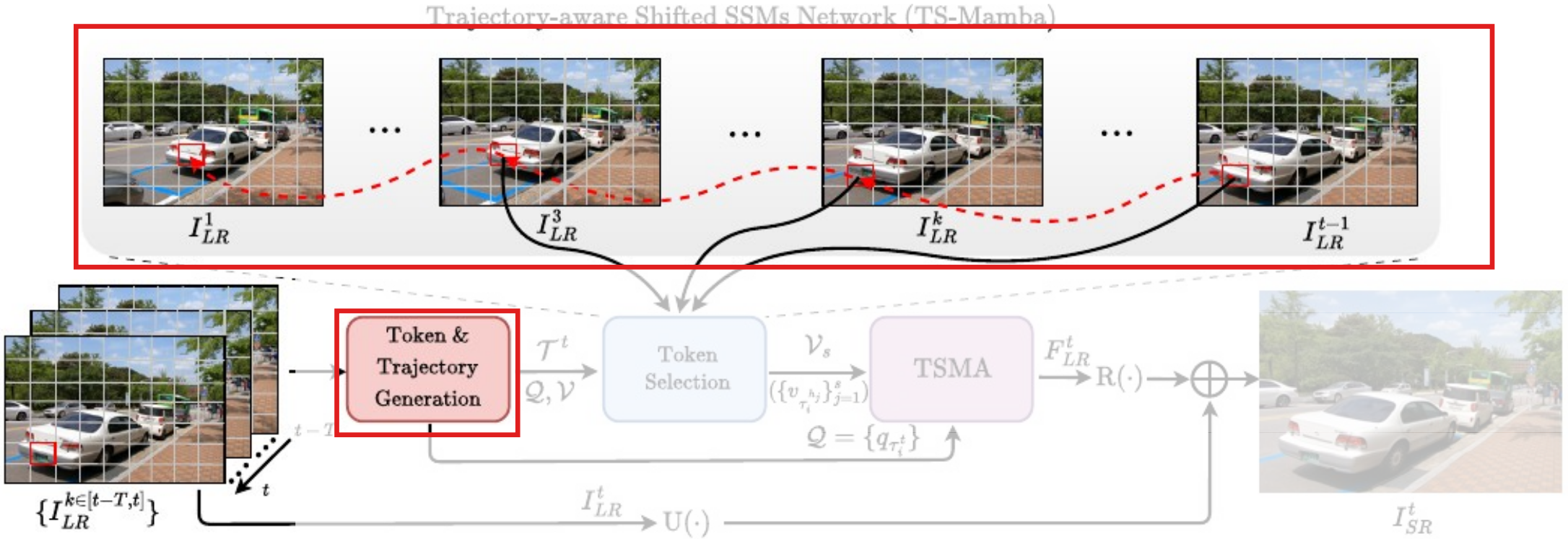
全体図：軌跡を作る → token を選ぶ → Mamba で集約 → 再構成



① 軌跡と token の生成　② 軌跡上で token を選ぶ　③ TSMA（Mamba で集約）　④ 再構成（R(·) ＋ bicubic）

TS-Mamba Fig. 2

Step 1：optical flow で、現在の各 token の軌跡を過去へたどる



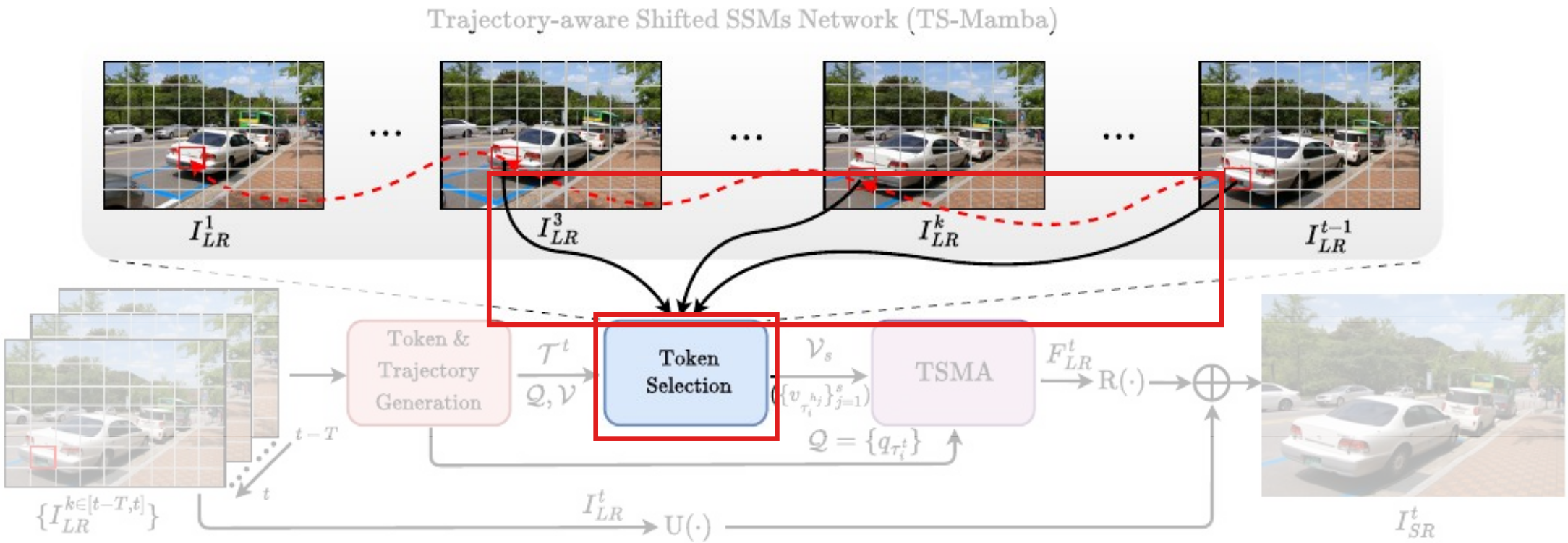
■ 軌跡：現在の token が、各過去フレームでどこにあったかの座標の列（終点が現在の位置）

■ 座標は軽量な optical flow（FastFlowNet）で更新（TTVSR に倣う）

■ 過去 15 フレーム、token は 4×4 画素

TS-Mamba Fig. 2, Eq. (3), Sec. 4.1

Step 2：軌跡上の過去 token から、似ている3個を選ぶ



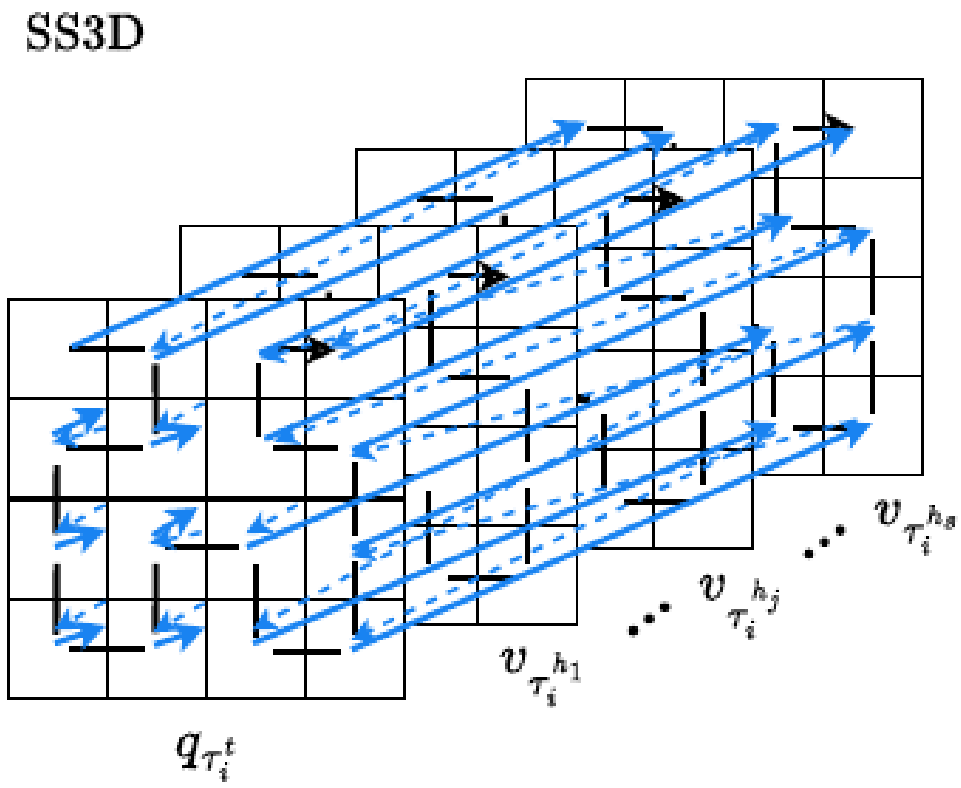
■ 現在の token と、軌跡上の過去 token の cosine 類似度を計算（Eq. 7）

■ 類似度の上位3個の**時刻**を選ぶ（空間方向の探索ではない）

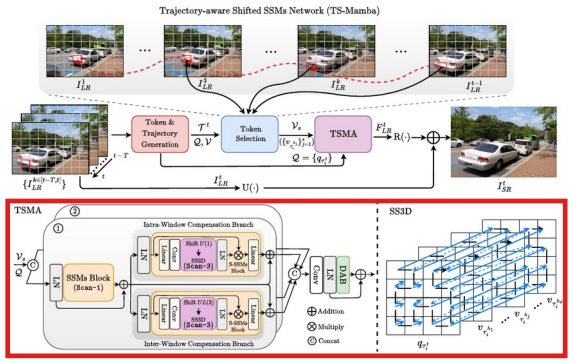
「どこに対応するか」＝ optical flow
「どの時刻を使うか」＝ 類似度
と役割を分けている

TS-Mamba Eq. (6)(7), Sec. 3.1, Sec. 4.1

Step 3：現在の token と選んだ token を並べて、Mamba で scan



SS3D：手前が現在の token、奥が選んだ token



全体図での位置（赤枠）

■ SS3D：現在の token と選んだ3個を、時空間の近傍が隣り合うように1列に並べて scan

■ TSMA：通常の SSM block ＋ shift した SSM block 2つ

■ フレームをまたいで持ち越す SSM の state はない

・毎フレーム、集めた token の系列の中で完結

・長期の情報は「過去の token そのもの」から取る

TV3S：state を持ち越す　↔　TS-Mamba：過去を集めて毎回流す

TS-Mamba Fig. 2, Sec. 3.2, 3.5

Ablation：軌跡を外すと最も下がり、選ぶ token は3個でほぼ頭打ち

Models	PSNR(dB)↑ / SSIM↑	Params.(M)↓	Run.(ms)↓	MACs(G)↓
(v1.1) *w/o* Trajectory	30.45 / 0.8678	1.7	20	84
(v1.2) *w/o* $\mathcal{L}_{trj}$	30.70 / 0.8721	3.0	29	112
(v1.3) *w/o* IntraWCB	30.58 / 0.8702	2.8	25	97
(v1.4) *w/o* InterWCB	30.61 / 0.8706	2.8	25	97
(v1.5) *w/o* IntraWCB+InterWCB	30.52 / 0.8689	2.4	21	85
(v1.6) *w/o* $U(1)/D(1)$	30.65 / 0.8710	3.0	27	112
(v1.7) *w/o* $UL(3)/DL(3)$	30.67 / 0.8714	3.0	27	112
(v1.8) *w/o* (v1.6) + (v1.7)	30.61 / 0.8702	3.0	25	111
TS-Mamba (ours)	30.73 / 0.8727	3.0	29	112

s	PSNR(dB)↑/SSIM↑	Params.↓ (M)	Run.↓ (ms)	MACs↓ (G)
1	30.64/0.8712	2.8	25	96
2	30.68/0.8720	2.9	27	104
3	30.73/0.8727	3.0	29	112
4	30.74/0.8727	3.1	31	120

■ 軌跡と token 選択を外す：30.73 → 30.45 dB

- params 3.0M → 1.7M、MACs 112G → 84G も同時に減る
- 軌跡と選択を別々に外した比較はない

■ 軌跡の loss だけ外す：−0.03 dB

■ 選ぶ数は3個でほぼ頭打ち（4個で +0.01 dB）

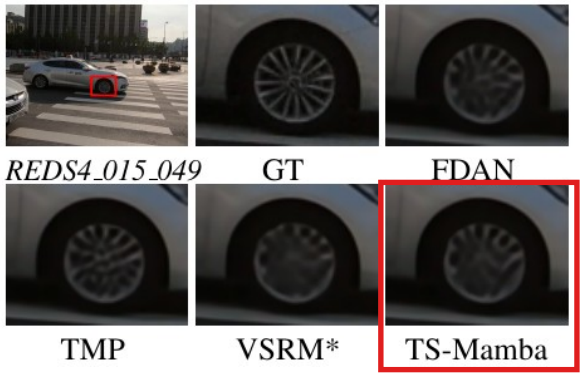
TS-Mamba Table 2, Table 3（REDS4、BI）

結果：PSNR は最良の手法と同等（+0.02 dB）で、計算量は少ない

オンライン手法の比較（Table 1 から抜粋、PSNR / SSIM）

手法	MACs (G)	REDS4（BI）	Vid4（BD）
RRN	193	28.82 / 0.8234	27.69 / 0.8488
BasicVSR++*	146	30.44 / 0.8686	27.49 / 0.8426
DAP-128	165	30.59 / 0.8703	—
FDAN	146	30.71 / 0.8723	27.76 / 0.8471
KSNet	145	30.69 / 0.8724	27.63 / 0.8444
TMP	176	30.67 / 0.8710	27.61 / 0.8428
VSRM*	136	30.64 / 0.8701	27.57 / 0.8423
TS-Mamba	112	30.73 / 0.8727	27.70 / 0.8473

- REDS4（BI）：**30.73** dB（FDAN 30.71、KSNet 30.69）
- MACs：112G（FDAN 146G、KSNet 145G）
- Vid4（BD）は FDAN 27.76 dB が上



限界（著者）：タイヤのような高速回転では軌跡が不正確になり、復元できない（赤枠：TS-Mamba）

TS-Mamba Table 1, Fig. 6, Sec. 5

まとめ：位置ではなく、動きの軌跡に沿って過去を集める



- TS-Mamba
  - optical flow で軌跡をたどり、軌跡上の過去 token から似ている3個を選ぶ
  - 現在の token と並べて Mamba で scan。state はフレームをまたいで持ち越さない
  - Mamba を替えた比較あり（付録 Table 4）：DCN などに替えると 30.59〜30.69 dB（TSMA 30.73）、MACs 132〜148G（TSMA 112G）
- 限界
  - 速い回転などで軌跡が崩れると復元できない。外れた軌跡を使わない仕組みはない
  - 軌跡の効果は、パラメータ数・計算量の変化と切り分けられていない

次論文：集めた過去や予測が外れていたら？
→ 外れたときは使わない仕組み（Memory as Dynamics）

Memory as Dynamics: Learning Reliability-Guided Predictive Models for Online Video Perception

SAM2 に物体の予測を足し、信頼できるときだけ使う

Minwoo Kim, Sang Min Yoon

ICML 2026

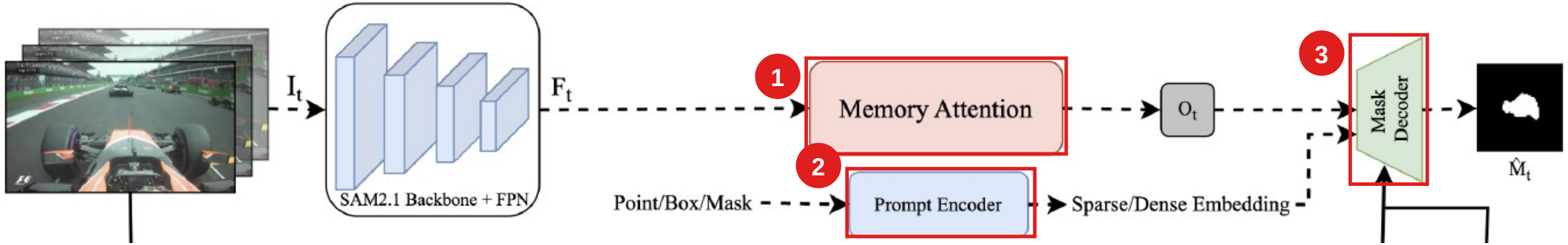
MambaOut
前提：Mamba は要るか

TV3S
位置に紐付けて
持ち越す

TS-Mamba
軌跡に沿って
集める

Memory as Dynamics
物体の予測を
信頼度で選ぶ

事前知識：SAM2 は過去フレームの結果を memory にためて次を分割する



■ 追跡する物体を最初のフレームで指定し、以降のフレームで mask を出す

■ ① memory attention：過去フレームの特徴と出力をためた memory bank を参照し、現在の特徴に混ぜる

■ ② prompt encoder：点・box・mask の指示を埋め込む

■ ③ mask decoder：mask と、その予測 IoU（どれだけ正しそうかの自己評価）を出す

■ 各フレームの出力は memory bank に追加される → 誤った出力も memory に入る

SAM 2 [Ravi+, ICLR2025] / 図：Memory as Dynamics Fig. 1（上段）, Sec. 3.1, 3.4

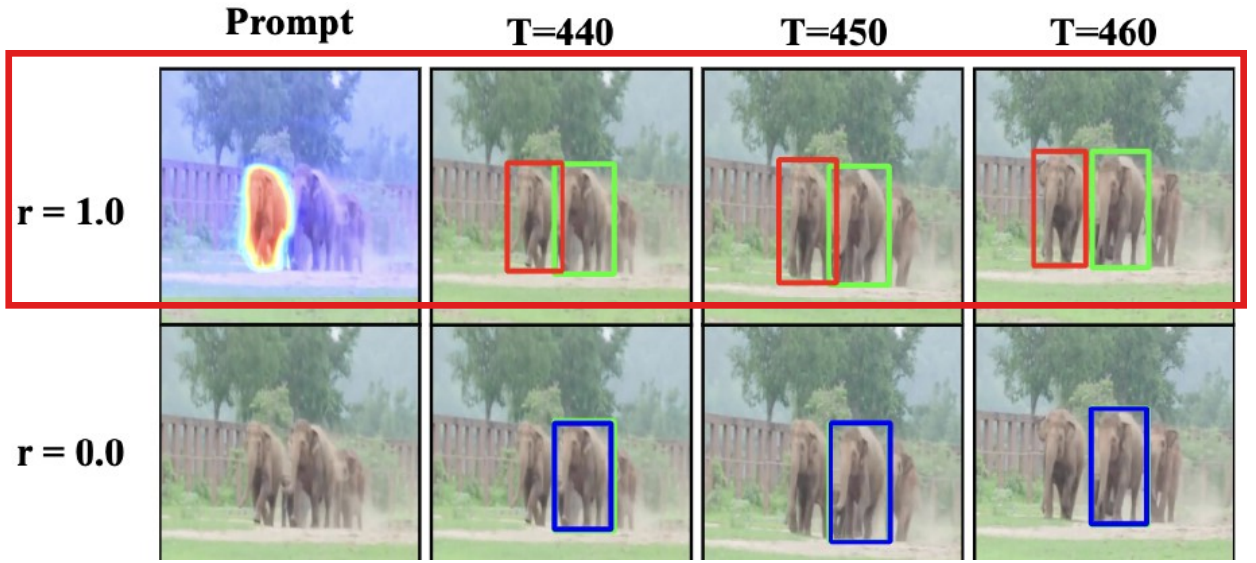
問題：予測を無条件に混ぜると、かえって drift する

■ SAM2 の memory（過去の観測）は、遮蔽・速い動き・紛らわしい物体に弱い

■ そこで未来を予測する memory を足したい

■ しかし予測を無条件に入れると、遮蔽中の誤った予測が memory に入り、**回復できない drift** を起こす

- 著者はこれを reliability mismatch と呼ぶ
- 「信頼できない予測も、観測と同じ重みで扱っている」



r = 1.0：予測を混ぜ続ける → 赤枠（追跡結果）が隣の象に移る

r = 0.0：予測なし → 青枠（正しい追跡）　緑枠：正解

Memory as Dynamics [Kim+, ICML2026] Abstract, Sec. 1, Fig. 6（Appendix B.4）

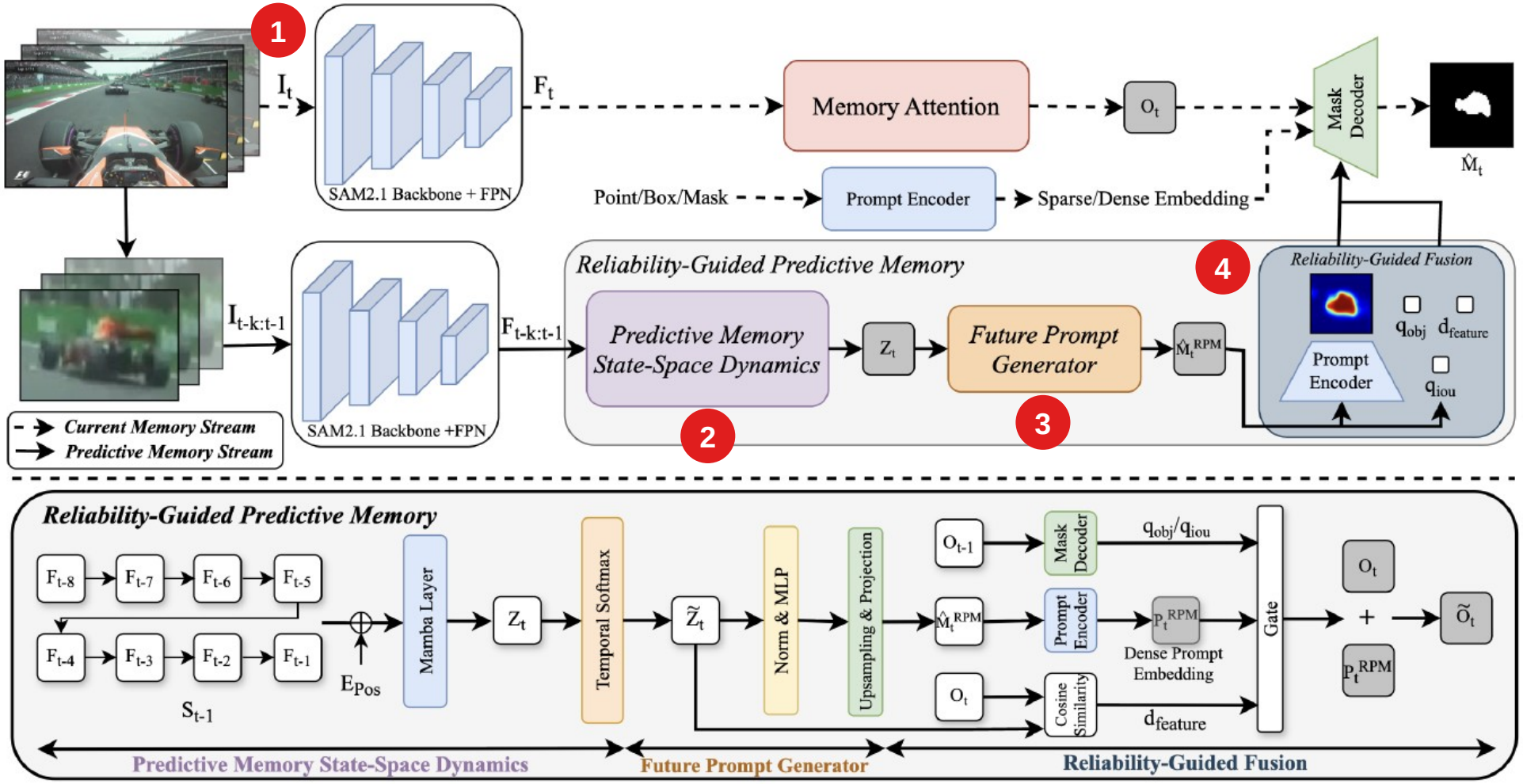
概要：予測は作るが、使うのは信頼できるときだけ（RPM）



- 前提：SAM2.1-B（固定）に HiM2SAM の memory 更新方式を組み合わせた構成の上に作る（全実験で共通）
- 提案：Reliability-Guided Predictive Memory（RPM）
  - 直近8フレームの特徴から、Mamba2 で次フレームの特徴を予測
  - 観測の信頼度が低く、かつ予測が観測と矛盾しないときだけ予測を使う
  - 予測は memory に直接は書き込まず、mask と同じ形の prompt（dense prompt）として decoder に足す
  - ただし予測を使ったフレームでは、その出力が memory に入る（Sec. 3.5）
- 学習：SAM2.1 は**完全に固定**。追加部分だけを VOS データで学習し、評価はトラッキングに zero-shot

Memory as Dynamics Sec. 1, 3.1, 3.5, 4.1, Appendix A.2

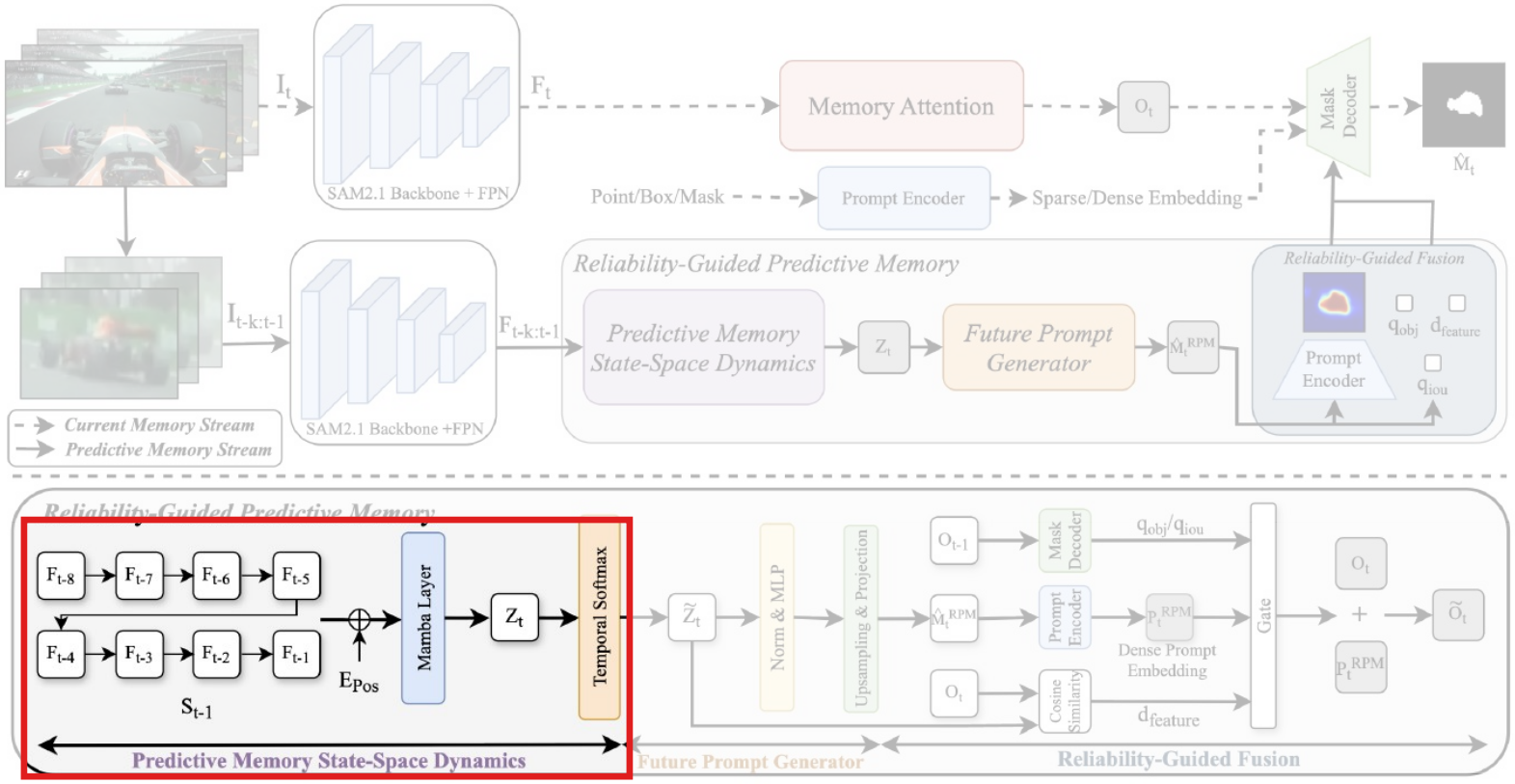
全体図：SAM2 の流れに、予測の流れと信頼度の判定を足す



① SAM2.1（固定）　② 予測（Mamba2）　③ 予測を mask と同じ形の prompt に変換　④ 信頼度で使うか決める

Memory as Dynamics Fig. 1

予測：直近8フレームの特徴を Mamba2 に通し、次フレームを出す



■ 入力：直近8フレーム分の特徴の列（時刻の埋め込みを足す）

■ Mamba2（24層）で8フレーム分を scan → 重み付きでまとめる

■ 1フレーム先だけを予測

■ **SSM の state は8フレームの窓の中だけ**

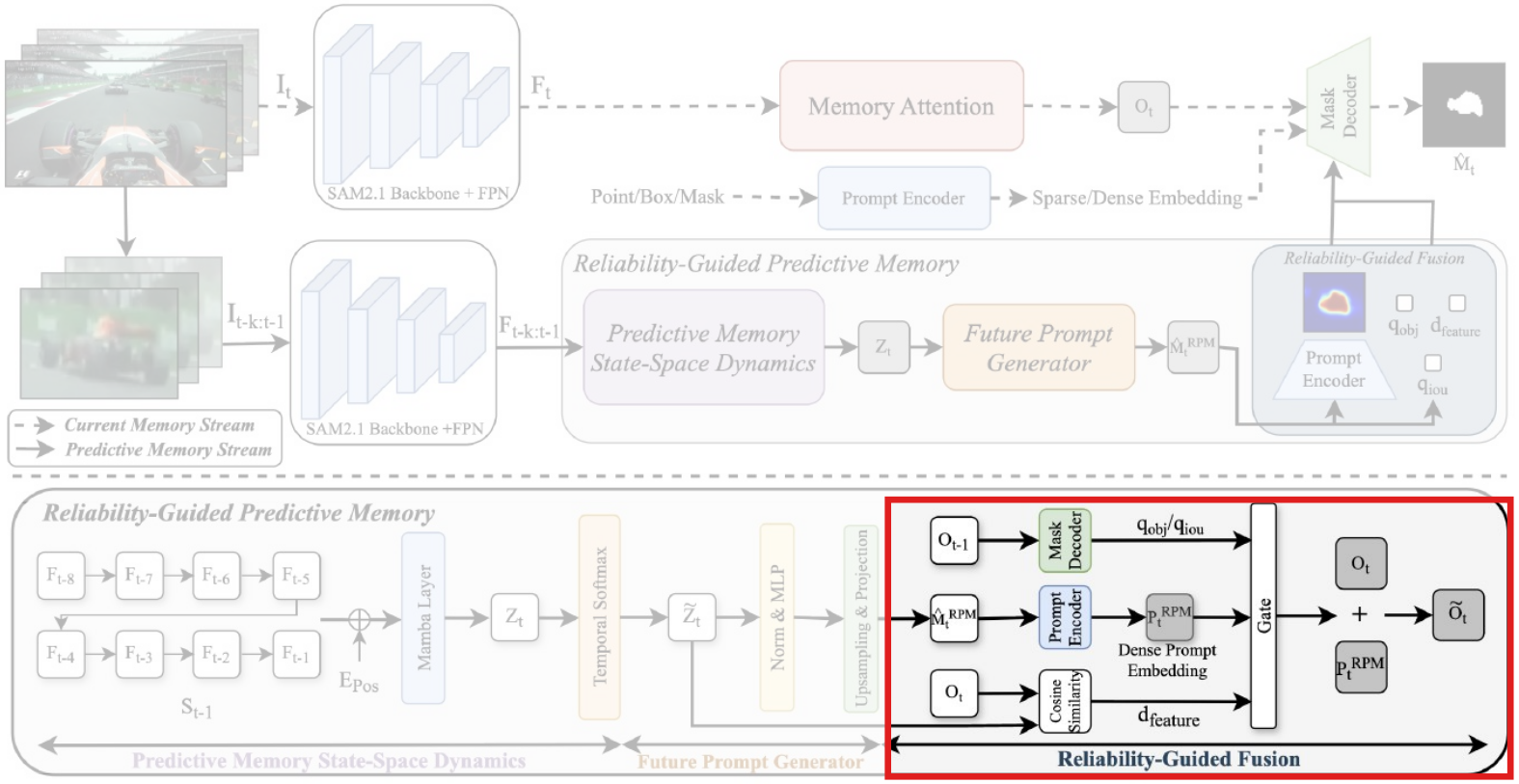
・毎時刻、窓を作り直す（持ち越さない）

・長く持ち越すのは SAM2 の memory bank

TS-Mamba と同じく、SSM は短い系列に使い、長期の情報は別の memory に置く

Memory as Dynamics Sec. 3.2, Eq. (4)–(6), Table 7

信頼度：観測が怪しく、予測が観測と矛盾しないときだけ使う



止めるのは「予測の使用」。Mamba の state 更新を止めるのではない

- 2段階の判定（Eq. 9）
  - ① SAM2 が出す前フレームの予測 IoU が 0.4 未満 → 予測を計算
  - ② 予測と観測の特徴の cosine 距離が 0.3 未満 → 予測を使う
- 使うときだけ decoder の入力に足す（Eq. 10）
  - $\tilde{o}_t = o_t + \rho_t \cdot p_t$（$\rho_t$ は 0 か 1）
- 判定は学習しない固定ルール
- 予測を使ったフレームの割合：**3.71%**（LaSOT）

Memory as Dynamics Sec. 3.4–3.5, Eq. (7)–(10), Table 7, Table 12

結果：zero-shot の SAM2 系 tracker で最も高い（LaSOT AUC 74.5）



Trackers	Source	#Param (M)	GOT-10k			LaSOT			LaSOT$_{ext}$		
			AO	$SR_{0.5}$	$SR_{0.75}$	AUC	P_{norm}	P	AUC	P_{norm}	P
SAMURAI-B (Yang et al., 2024)	arXiv'24	81	79.6	**90.8**	72.9	70.7	78.7	76.2	57.5	69.3	67.1
SAM2.1-B (Ravi et al., 2025)	ICLR'25	81	77.9	88.6	71.5	66.0	73.5	71.0	55.5	67.2	64.6
DAM4SAM-B (Videnovic et al., 2025)	CVPR'25	81	78.1	88.5	70.9	73.3	81.3	78.8	58.6	69.4	68.2
HiM2SAM-B (Chen et al., 2025a)	PRCV'25	81	78.1	88.9	71.7	73.4	81.7	79.5	59.3	71.2	69.4
Ours-B	ICML'26	93	**79.7**	90.6	72.9	74.5	82.9	80.7	**60.9**	**73.1**	**71.4**

- LaSOT AUC：SAM2.1-B 66.0 → **74.5**、LaSOT$_{ext}$ AUC：55.5 → 60.9
- ただし HiM2SAM の memory 更新方式を下敷きにしている
  - RPM の上乗せの目安は HiM2SAM-B との差（LaSOT 73.4 → 74.5、LaSOT$_{ext}$ 59.3 → 60.9）
- 追加部分を VOS データで学習し、パラメータは 81M → 93M
- VOS（J&F）でも SAM2.1 より高い（SA-V test 77.7 → 79.8）

Memory as Dynamics Table 1, Table 6, Sec. 4.1

Ablation：予測を毎フレーム使うと 70.5、0/1 で切り替えると 74.5

Method	AUC
SAM2.1-B	66.0
Always-on Prediction Injection	70.5
Soft-Gating (Learned Scalar)	72.3
Reliability-Guided Binary Gating	**74.5**

参考：予測なしの HiM2SAM-B は 73.4（Table 1）。ただし同じ構成かは論文に明記がない

Trackers	Short(<5)	Medium(5–20)	Long(>20)
SAMURAI	0.498	0.421	0.311
HiM2SAM	0.552	0.482	0.384
Ours	**0.577**	**0.517**	**0.421**

■ 混ぜ方の比較（LaSOT AUC）
- 毎フレーム予測を足す：70.5
- 学習した重みで少しずつ混ぜる（soft）：72.3
- 信頼度で 0/1 に切り替える：**74.5**

■ 遮蔽が長いほど差が開く（遮蔽明け5フレームの IoU）
- 20フレーム超の遮蔽：HiM2SAM 0.384 → 0.421

Memory as Dynamics Table 14（Appendix B.4）, Table 5, Table 1

まとめ：長期の memory は SAM2 に任せ、予測は信頼できるときだけ使う



- Memory as Dynamics（RPM）
  - 直近8フレームから Mamba2 で次フレームを予測し、prompt として足す
  - 観測の信頼度と予測の一貫性で、使うかどうかを 0/1 で切り替える
  - LaSOT では、0/1 で切り替える方が少しずつ混ぜるより良かった（74.5 vs 72.3）
- 限界・注意
  - RPM 単体の効果が分離されていない（HiM2SAM の memory 更新が下敷き）
  - 信頼度の効果の大半は予測 IoU 単独（70.5 → 73.8）
  - Mamba を他の時系列モデルに替えた比較はない。複数物体の扱いは書かれていない

本紹介のまとめ：過去の情報を何に紐付け、どこまで信じるか



前提（MambaOut）：SSM の state は固定サイズで lossy。長く causal な系列で使う

論文（タスク）	紐付け先	SSM が scan する系列	長く持ち越すもの	信じる仕組み	Mamba を替えた比較
TV3S（動画 semantic seg）	位置（20×20 window）	window 内の token と過去フレーム	window ごとの SSM state	明示的な仕組みなし（入力依存の更新のみ）	なし
TS-Mamba（オンライン動画超解像）	軌跡（optical flow）	軌跡上の現在＋過去3 token	過去15フレームの token と軌跡	類似度で使う時刻を選ぶ	あり（付録）+0.04〜0.14 dB
Memory as Dynamics（SOT / VOS）	物体（SAM2 の追跡対象）	直近8フレーム分の特徴	SAM2 の memory bank	信頼度で予測を 0/1 に切り替える	なし

・タスクが違うので、3本の優劣は比べられない
・SSM の state を長く持ち越すのは TV3S だけ。TS-Mamba と MaD は長期の情報を別の memory に置き、SSM は短い系列に使う
・MaD（LaSOT）では、予測を 0/1 で切り替える方が少しずつ混ぜるより良かった（他の条件では未検証）



補足

補足（TV3S）：推論は1フレームずつ、state を次へ渡す

Algorithm 1 Inference Procedure for TV3S Architecture

1: **Input:** Frame sequence $\{\boldsymbol{I}_t\}$
2: **Output:** Segmentation map $\boldsymbol{O}_t$
3: **Initialize:** Hidden states $\boldsymbol{H}$ for all TV3S blocks
4: **for** each frame $\boldsymbol{I}_t$ in the sequence **do**
5: $\quad \boldsymbol{E}_t \leftarrow \text{Encoder}(\boldsymbol{I}_t)$ // Extract features
6: $\quad$ **for** each patch (i, j) in $\boldsymbol{E}_t$ **do**
7: $\quad\quad \hat{\boldsymbol{P}}^{i,j} \leftarrow \text{Flatten}(\boldsymbol{E}_t[i : i + w, j : j + w, :])$
8: $\quad\quad$ **for** each TV3S block n from 1 to N **do**
9: $\quad\quad\quad$ //Update Features and Hidden State.
10: $\quad\quad\quad \boldsymbol{F}_t^{n,i,j}, \boldsymbol{H}_t^{n,i,j} \leftarrow \text{TV3S}^n(\hat{\boldsymbol{P}}^{i,j}, \boldsymbol{H}^{n,i,j})$
11: $\quad\quad$ **end for**
12: $\quad$ **end for**
13: $\quad \boldsymbol{O}_t \leftarrow \text{Interpolate}(\text{Linear}(\boldsymbol{F}_t^N))$ // Segmentation
14: $\quad \boldsymbol{H} \leftarrow \{\boldsymbol{H}_t\}$ // Store for next frame
15: **end for**

- Algorithm 1（推論）
  - 10行目：window ごとに state を更新
  - 14行目：更新した state を次のフレームのために保存
- 動画全体の過去を state 経由で使う（オンライン）
- 学習は4フレーム（t−9, t−6, t−3, t）
  - 学習と推論で系列長が違う点は、論文では検証されていない（発表者の所見）

TV3S Algorithm 1, Sec. 3.3–3.4, Sec. 4.1

補足（TV3S）：実験設定



- データセット：VSPW
  - 124 クラス、15fps で密にアノテーション
  - train 2,806 clips / val 343 clips / test 387 clips
- 評価指標
  - mIoU：各フレームのセグメンテーション精度
  - mVC_n：**連続 n フレームで正解と一致し続ける領域の割合**（n = 8, 16）
    - 例：8フレーム続けて「人」と正しく塗れていた画素の割合 → ちらつきが少ないほど高い
- backbone：MiT-B1 など（ImageNet 事前学習、全体を end-to-end で学習）

TV3S Sec. 4.1, Eq. (4)

補足（TV3S）：shift で隣と混ぜると +0.6 mIoU（38.9 → 39.5）

Models	Evaluation (mIoU)		
	Add	Concat	Direct
1 TSS (No Shift)	37.6	37.8	38.0
TV3S (No Shift)	38.1	38.4	38.9
TV3S (Shift)	**38.5**	**39.3**	**39.5**

位置に固定した state でも、隣と混ぜれば少し良くなる

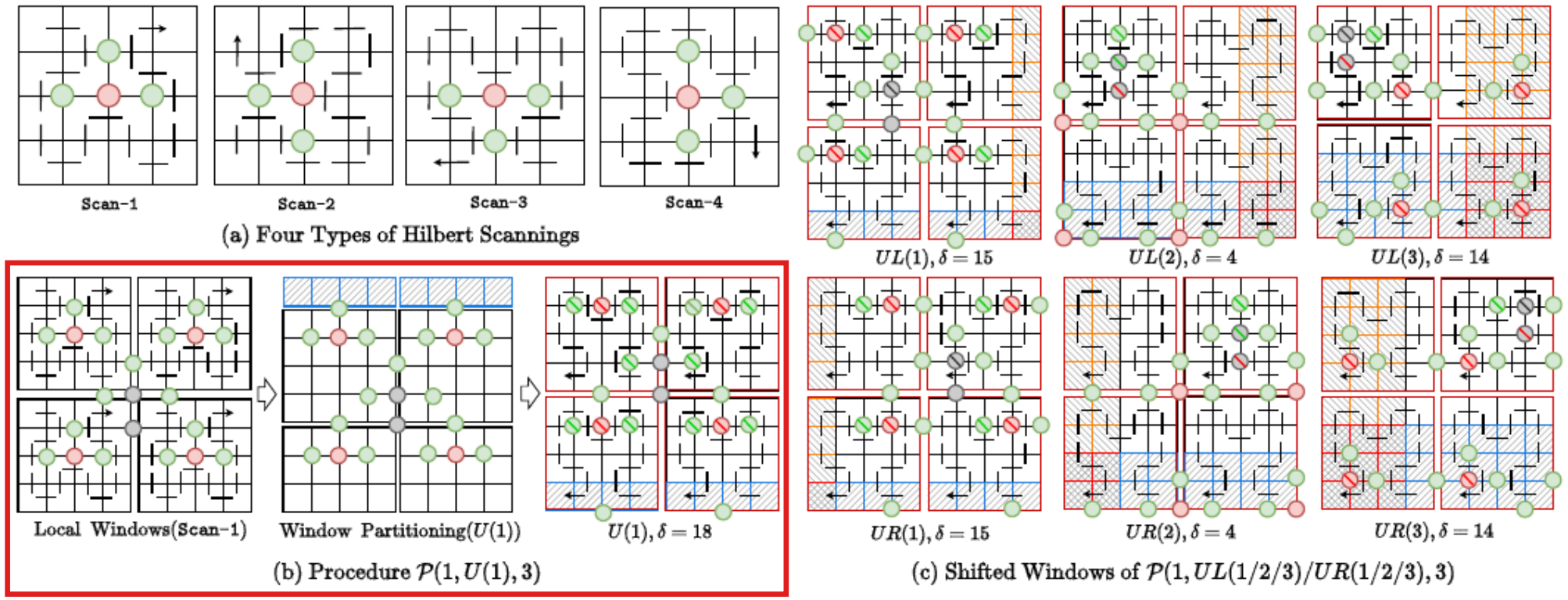
- Table 3（MiT-B1、Direct 列）
  - 1 TSS（shift なし）：38.0
  - TV3S（shift なし）：38.9
  - TV3S（shift あり）：**39.5**
- 3行とも state は持ち越している
  - 比べているのは shift の有無と出力のつなぎ方

TV3S Table 3, Sec. 4.3

補足：TV3S の VSPW 全結果（Table 1）

Methods	Backbones	mIoU↑	mVC_8↑	mVC_{16}↑	GFLOPs↓	Params(M)↓	FPS↑
Segformer† [69]	MiT-B1	36.5	84.7	79.9	26.6	13.8	58.7
CFFM [57]	MiT-B1	38.5	88.6	84.1	-	15.5	29.8
MRCFA [58]	MiT-B1	38.9	88.8	84.4	-	16.2	40.1
TV3S (Ours)	MiT-B1	**40.0**	**90.7**	**87.0**	36.9	17.3	24.7
Segformer [69]	MiT-B2	43.9	86.0	81.2	100.8	24.8	16.2
CFFM [57]	MiT-B2	44.9	89.8	85.8	143.2	26.5	10.1
MRCFA [58]	MiT-B2	45.3	90.3	86.2	127.9	27.3	10.7
TV3S (Ours)	MiT-B2	**46.3**	**91.5**	**88.35**	53.9	28.3	21.9
Mask2Former† [7]	R50	38.5	81.3	76.4	110.6	44.0	19.4
MPVSS [67]	R50	37.5	84.1	77.2	38.9	84.1	33.9
Mask2Former† [7]	R101	39.3	82.5	77.6	141.3	63.0	16.9
MPVSS [67]	R101	38.8	84.8	79.6	45.1	103.1	32.3
DeepLabv3+† [6]	R101	34.7	83.2	78.2	379.0	62.7	9.2
UperNet† [68]	R101	36.5	82.6	76.1	403.6	83.2	16.0
PSPNet† [72]	R101	36.5	84.2	79.6	401.8	70.5	13.8
OCRNet† [71]	R101	36.7	84.0	79.0	361.7	58.1	14.3
TCB† [42]	R101	37.8	87.9	84.0	1692	-	-
ETC† [37]	OCRNet	37.5	84.1	79.1	361.7	-	-
Segformer [69]	MiT-B5	48.9	87.8	83.7	185.0	82.1	9.4
CFFM [57]	MiT-B5	49.3	90.8	87.1	413.5	85.5	4.5
MRCFA [58]	MiT-B5	**49.9**	90.9	87.4	373.0	84.5	5.0
TV3S (Ours)	MiT-B5	49.8	**91.7**	**88.7**	137.0	85.6	14.0
Mask2Former† [7]	Swin-T	41.2	84.5	80.0	114.4	47.4	17.1
MPVSS [67]	Swin-T	39.9	85.9	80.4	39.7	114.0	32.8
TV3S (Ours)	Swin-T	**44.9**	**88.0**	**83.5**	57.3	31.7	22.9
Mask2Former† [7]	Swin-S	42.1	84.7	79.3	152.2	68.9	14.5
MPVSS [67]	Swin-S	40.4	86.0	80.7	47.3	108.0	30.6
TV3S (Ours)	Swin-S	**50.6**	**89.6**	**85.8**	94.1	53.1	19.5

補足（TS-Mamba）：1列に並べると切れる隣接を、shift で補う



■ 2次元を1列に並べると、隣同士なのに離れて読まれる場所（不連続）が出る → Hilbert 順に並べ、window をずらして別の順で2回目の scan をかける

■ 赤枠：上に1つずらす shift（記号 U(1)）→ Scan-3 が window 内の不連続を最も多く解消（δ = 18）。window 間は UL(3)

TS-Mamba Fig. 3, Sec. 3.3–3.4（δ：shift で解消できた不連続の数）

補足：TS-Mamba の全比較（Table 1）

Category	Methods	Supp. Frame	R-T	Run.↓ (ms)	FPS↑ (1/s)	MACs↓ (G)	Params.↓ (M)	BI degradation REDS4(RGB)↑ (PSNR/SSIM)	BI degradation Vid4(Y)↑ (PSNR/SSIM)	BD degradation Vimeo-90K-T(Y)↑ (PSNR/SSIM)	BD degradation Vid4(Y)↑ (PSNR/SSIM)
Bidirectional	BasicVSR	P+F	✗	63	15.9	397	6.3	31.42/0.8909	27.24/0.8251	37.53/0.9498	27.96/0.8553
	IconVSR	P+F	✗	70	14.3	452	8.7	31.67/0.8948	27.39/0.8279	37.84/0.9524	28.04/0.8570
	BasicVSR++	P+F	✗	77	13.0	418	7.3	32.39/0.9069	27.79/0.8400	38.21/0.9550	29.04/0.8753
	SSL-bi	P+F	✗	24	41.7	92	1.0	31.06/0.8933	27.15/0.8208	37.06/0.9458	27.56/0.8431
	DFVSR	P+F	✗	-	-	-	7.1	32.76/0.9081	27.92/0.8427	38.51/0.9571	29.56/0.8983
	MIA-VSR	P+F	✗	318	3.1	3220	16.5	32.78/0.9220	28.20/0.8507	-	-
	IART	P+F	✗	180	5.6	5020	13.4	32.90/0.9138	28.26/0.8517	38.62/0.9579	29.68/0.8884
	VSRM	P+F	✗	223	4.5	2174	17.1	33.11/0.9162	28.44/0.8552	-	-
Online	Bicubic	N	✓	-	-	-	-	26.14/0.7292	23.78/0.6347	31.30/0.8687	21.80/0.5246
	RRN	P	✓	34	29.4	193	3.4	28.82/0.8234	25.85/0.7660	36.69/0.9432	27.69/**0.8488**
	BasicVSR++*	P	✓	40	25.0	146	**3.0**	30.44/0.8686	27.06/0.8173	37.11/0.9464	27.49/0.8426
	DAP-128	P	✓	38	26.3	165	-	30.59/0.8703	-	37.29/0.9476	-
	FDAN	P	✓	34	29.4	146	3.9	30.71/0.8723	27.14/0.8206†	**37.36**/0.9483†	**27.76**/0.8471
	KSNet	P	✓	31	32.3	145	**3.0**	30.69/0.8724	27.14/0.8208	37.34/**0.9490**	27.63/0.8444†
	TMP	P	✓	**25**	**40.1**	176	3.1	30.67/0.8710	27.10/0.8167	37.33/0.9481	27.61/0.8428
	VSRM*	P	✓	31	32.7	136	3.1	30.64/0.8701	27.10/0.8163	37.28/0.9477	27.57/0.8423
	TS-Mamba	P	✓	29	33.5	**112**	**3.0**	**30.73/0.8727**	**27.17/0.8209**	**37.36**/0.9482	27.70/0.8473

補足：Memory as Dynamics の信頼度の内訳と予測を使った割合

q_{iou}	q_{obj}	d_{feature}	AUC	P_{norm}	P
×	×	×	70.5	78.7	76.6
✓	×	×	73.8	82.0	79.8
✓	✓	×	73.9	82.1	79.9
✓	×	✓	74.0	82.2	80.1
✓	✓	✓	**74.5**	**82.9**	**80.7**

- q_{iou}（予測 IoU）だけで 70.5 → 73.8（上がり幅の大半）
- 3つすべてで 74.5
- q_{obj} は、予測を置く位置の判定に使う
- 予測を使ったフレームの割合：LaSOT 3.71%、$\text{LaSOT}_{\text{ext}}$ 4.89%

Dataset	Target Unobservability (%)		Low-Confidence Observation (%)	Activation Rate (%)
	Full-Occlusion	Out-of-View		
LaSOT	1.14	1.54	1.03	3.71
LaSOT_{ext}	3.23	0.28	1.38	4.89

Memory as Dynamics Table 3, Table 12, Sec. 3.4–3.5

補足：MambaOut の「長い系列」の目安（Sec. 3.2）



- Transformer block の FLOPs = $24D^2L + 4DL^2$（Eq. 6）
- L の2乗の項が1次の項を超えるのは $L > 6D$ のとき（Eq. 7）
  - ViT-S（D = 384）：$6 \times 384 = 2304$ ／ ViT-B（D = 768）：4608
- ImageNet（224^2、patch 16^2）：$14^2 = 196$ token → 長くない
- COCO（800 × 1280）・ADE20K（512 × 2048）：約 4K token → 長い